Anteater Software Spec Manual

Version 1.0

Authors: Prithvi Binu, Justin Wu, Katrina Gong, William Culley, Stefano Pinna Segovia, Rishab Senthil Kumar

Affiliation: University of California, Irvine
Team Name: Blunder Boys + Katrina

Table of Contents

Table of Contents....2
Glossary.... 3
1. Software Architecture Overview.... 5
1.1 Main data types and structures....5
1.2 Major software components....6
1.3 Module interfaces.... 6
1.4 Overall program control flow....7
1. Determine gamemode (Human vs Human, Human vs AI, AI vs AI)....7
2. Initialize board setup....7
2. Installation.... 7
2.1 System requirements and compatibility....7
2.2 Setup and configuration....8
2.3 Building, compilation, and installation....8
3. Documentation of Packages, Modules, and Interfaces....8
3.1 Detailed description of data structures.... 8
3.2 Detailed description of functions and parameters....10
3.3 Detailed description of input and output formats.... 14
4. Development Plan and Timeline....14
4.1 Partitioning of Tasks....14
4.2 Team member responsibilities.... 14
Copyright.... 15
Error Messages.... 15
Index.... 15
References.... 16

Glossary

Term	Definition
Ant	Another name for a Pawn which is the basic piece that can only move one tile forward unless it has not moved yet. In that case, it would be able to move 2 tiles forward. An Ant is able to promote into any other piece (other than another Ant or King), when it reaches the opposite side of the chessboard.
Ant eating	A special move by the Anteater that can capture multiple Ants at once. The move is initiated by the Anteater capturing the first Ant. If there are any adjacent tiles with Ants, it will target that direction until there are no Ants left, provided that they are at the same rank or file.
Bishop	A piece that moves diagonally across the board any number of spaces.
Castling	A special move that can be played once by each player. This move allows the king to move left or right two tiles to swap places with the Rook on the kingside or queenside. Both the Rook and the King may not have moved a single tile before the attempt to castle. No pieces may be in between them while castling.
Check	When a player's King is vulnerable to capture on the opponent's next turn.
Checkmate	When a player's King is under check and cannot escape.
En Passant	A special pawn capture move that means "in passing". This move starts when an Ant moves two tiles forward from its initial position to attempt to avoid capture from an enemy Ant. The enemy Ant is allowed to capture it as if it had only moved one tile forward.
File	The vertical columns on a chessboard,

	labeled alphabetically (A-J).
King	The most important piece and target for attack. Moves one square in any direction. The game ends when the king is in checkmate.
Knight	A piece that moves in an L-shape, two squares in one direction and one square perpendicular. The only piece that can jump over other pieces.
Legal moves	The list of moves allowed for a piece. The King's condition is relevant. If a player's king is under check, a move that is played must get the King out of that condition.
Pawn	A piece that moves forward one square, or two squares from its starting position, and may only capture diagonally. Also called an ant.
Promotion	When a pawn reaches the opposite end of the board, it is promoted to another piece
Queen	The most powerful piece, moves in any direction, horizontally, vertically, and diagonally any number of square.
Piece	Items that are moved on a chessboard, each with their unique movesets (King, Queen, Rook, Bishop, Knight, Ant, Anteater).
Rank	The horizontal rows on a chessboard, labeled numerically (1-8).
Rook	A piece that moves horizontally or vertically any number of squares.
Stalemate	When a player's King is not under check, but they are not able to make a legal move, resulting in a draw.

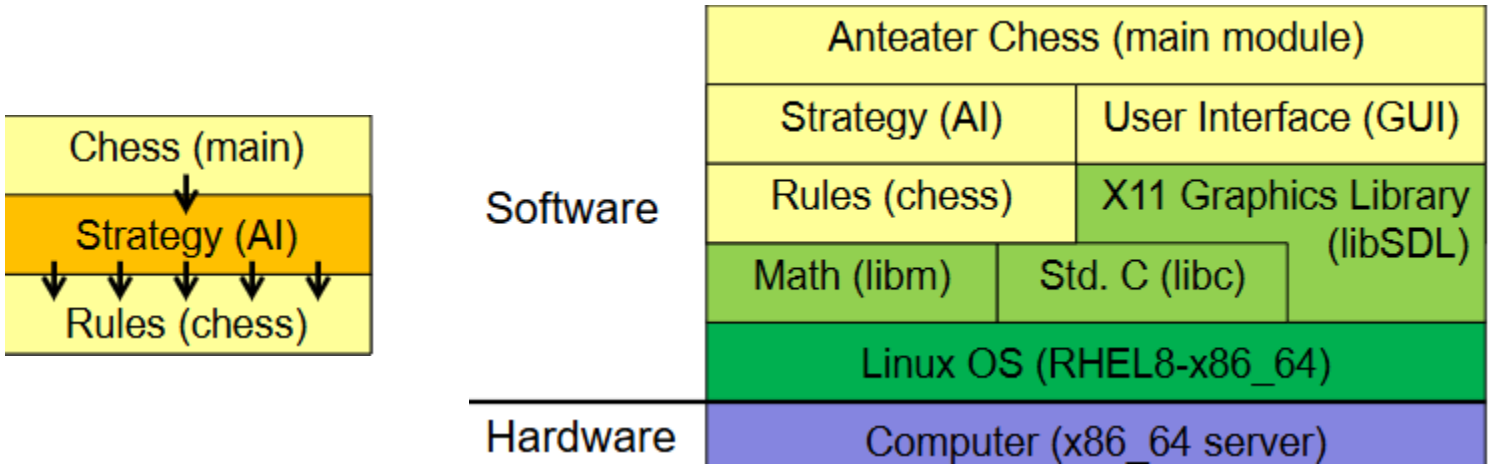
1. Software Architecture Overview

1.1 Main data types and structures

1. Struct Player -
    a. Contains: name, color, isHuman
2. typedef enum Piece type
    a. Contains: ant, knight, bishop, rooke, queen, king, and anteater
3. typedef enum Piece color
    a. Contains: white and black
4. Piece - struct *
    a. Contains: piece type and piece color enums
5. struct piece * array Board
    a. Contains: 2D w/ NUM_RANKS=8 & NUM_FIL ES=10
6. Position of piece - struct *
    a. Contains: Rank = char, File = char
7. Struct location
    a. Contains: Rank = int, File = int
8. Move - struct
    a. Pos1, pos 2
9. Log of moves - struct { int move, int piece, int specialMoves, history[1000]}
    a. Contains:
        i. Moves - char[]
        ii. Piece - struct piece w/ pieceColor and pieceType
        iii. moveNumber - int
        iv. moveHistory - char[]
10. Struct GameState
    a. Stores the following:
        i. Board
        ii. currentPlayer
        iii. enPassantTiles
        iv. CastlingRights
        v. halfMove_count ←for 50 move draw rule
        vi. fullMove_count
        vii. Log of moves
    b. This struct is useful for tracking special rules like castling or En Passant, and storing the board and log of moves.
    c. Functions that make changes to the board should interact with the GameState by reference (gameState*).
    d. Functions that only access the GameState's attributes should use const reference(const gameState*).

1.2 Major software components

- Diagram of module hierarchy



1.3 Module interfaces

This program is split into four modules.

1. GUI Module
    a. Provides the user interface for playing the game of chess or simulating it against the engine. The starting color can be chosen by the user. The gamemode can be chosen. If the user chooses to include the engine, the difficulty level (Easy, Medium, Hard) is set in a drop down menu.
    b. Displays the board and provides buttons for the user to move pieces.
    c. The user is able to make a legal move by dragging a button icon of a piece onto another tile on the board.
    d. The user is able to undo moves with the undo button, and create new games with the new game button.
    e. The user is able to view a move timer, a 50 move draw tracker, as well as the move log.
2. Moves/Rules Module
    a. Provides the API for move generation.
    b. The most important functions are getMoves and getPsuedoLegalMoves which return all legal moves or all pseudolegal moves.
    c. Adds a function to apply a move and undo a move.
    d. Verifies check and checkmate conditions.
    e. Used by both the GUI for correct button placement and the engine for a list of all legal positions.
3. Engine Module
    a. Provides the API for the computer to move.

        b. The main functions are negaMax with alpha-beta pruning, move ordering, and quiescence search. There are also wrappers to limit the time per move.
        c. This module takes the static game state and evaluates it to a certain depth to generate the best possible move within an allotted time.
    4. Evaluation Module
        a. Provides the functions for the engine to score certain positions.
        b. NegaMax and several other functions require some sort of score to pick the best moves.
        c. Determines the favorability of moves by checking the material weight, center control, and specific move behavior like anteating or castling.
        d. Gives bonus value to favorable moves like captures and gives negative value to moves such as hanging a piece.

The evaluation module gives a way for the engine to know what a good move is by giving it a score. The engine can know what legal moves are possible for all positions based on the moves/rules module. The engine and rules modules are used by the GUI to handle the legal interactions for both human and computer gameplay.

1.4 Overall program control flow

1. Determine gamemode (Human vs Human, Human vs AI, AI vs AI)
    a. Choose which side to start
    b. Choose the difficulty of the engine
2. Initialize board setup
    a. Setup all objects including board, pieces, and players
    b. Display board using GUI
3. Repeat following steps
    a. Prompt User for Move
    b. Check for legality
    c. White player makes a move
    d. Display the board
    e. If black is in checkmate, white wins!
    f. Black player makes a move
    g. Display the board
    h. If white is in checkmate, black wins

2. Installation

2.1 System requirements and compatibility

- C Compiler with C11 Support (like gcc)
- GTK+ 3 libraries for use of GUI

- This program is built for Linux systems.

2.2 Setup and configuration

1. Download the user package from the server
   >> git clone team16@zuma.eecs.uci.edu:repos/chess.git
2. Extract the user package using the following commands:
   % tar -xvzf Chess_V1.0_src.tar.gz
   % cd Chess_V1.0_src

- The above commands allow the user to extract the chess folder using [tar] and open the folder.

2.3 Building, compilation, and installation

1. Run makefile to compile and build all modules using:
   % make
2. Run the program after compilation:
   % ./bin/chess

3. Documentation of Packages, Modules, and Interfaces

3.1 Detailed description of data structures

```
enum pieceType {
    KING, QUEEN, KNIGHT, BISHOP, ROOK, ANT, ANTEATER
};

enum pieceColor {
    BLACK, WHITE
};

enum gameMode {
    HUMAN_VS_HUMAN,
    HUMAN_VS_AI,
    AI_VS_AI
};

struct piece {
```

```
    enum pieceType  piece;
    enum pieceColor color;
};

typedef uint32_t Move;

struct log {
    Move move; // encoded move (first 7 bits are fromTile,
next 7 bits are toTile)
    int piece; // Black or White
    int moveNumber; // 1-1000 for # of moves in the game
    int history[1000];
};

struct gameState {
    struct piece* board[8][10];
    enum pieceColor currentPlayer;
    int whiteAntCount;
    int blackAntCount;

    bool whiteKingMoved;
    bool blackKingMoved;
    bool whiteRookMovedQS;   /* queenside rook — column A
(index 0) */
    bool whiteRookMovedKS;   /* kingside  rook — column J
(index 9) */
    bool blackRookMovedQS;
    bool blackRookMovedKS;
    int  enPassantCol;       /* -1 if unavailable
*/
    int  enPassantRow;       /* row of the double-advanced ant
*/

    //trackers for move log + 50 turn draw rule
    int halfMove_count;
    int fullMove_count;
    struct log move_log;
```

```
};
```

Move Structure:

A move will be encoded within a 32 bit unsigned integer. Using bitwise operators, the tile column, row, and flags can be extracted easily. Additionally, packing this information into a smaller 4 byte integer allows cheaper copies than if a large struct was used.

fromRow [4 BITS]	fromCol [4 BITS]	toRow [4 BITS]	toCol [4 BITS]	MOVE FLAGS [16 bits]

This is a list of the flags used. Without flags, the board would have to be queried many times to determine the type of move. It would be much more difficult to determine if it's a normal move or a special move like En Passant.

```
/* — Move flags ————————————————
#define MOVE_NORMAL         0
#define MOVE_EN_PASSANT     1
#define MOVE_CASTLE_KS      2
#define MOVE_CASTLE_QS      3
#define MOVE_PROMO_QUEEN    4
#define MOVE_PROMO_ROOK     5
#define MOVE_PROMO_BISHOP   6
#define MOVE_PROMO_KNIGHT   7
#define MOVE_PROMO_ANTEATER 8
#define MOVE_ANTEATING      9
```

3.2 Detailed description of functions and parameters

- Function prototypes and brief explanations
- int getScore(const struct gameState* gs);
    - Main evaluation function for static positions.
    - Uses material count, ant advancement, anteater value scaling, development, king safety, center control, hanging pieces, and mobility.
    - Parameters:
        - gs: current game state
    - Returns:
        - Evaluation score from the side-to-move perspective
- static int negaMax(struct gameState* gs, int depth, int alpha, int beta, int ply);
    - Recursive function that returns the best score(from getScore)
    - Uses negamax with alpha-beta pruning to evaluate future positions.
    - Also uses move ordering, repetition checks, and quiescence search.
    - Parameters:
        - gs: current game state
        - depth: remaining search depth
        - alpha: lower bound of the search window
        - beta: upper bound of the search window
        - ply: current distance from the root of the search tree
    - Returns:
        - Evaluation score for the position

- uint32_t depthSearch(struct gameState* gs, int depth, uint32_t pvMove, int alpha, int beta, int* outputScore);
    - Searches all moves at one depth and returns the best move.
    - Uses the previous best move first to improve ordering.
    - Parameters:
        - gs: current game state
        - depth: search depth
        - pvMove: previously best move from a shallower iteration
        - alpha: lower search bound
        - beta: upper search bound
        - outputScore: stores the final score of the chosen move
    - Returns:
        - Best encoded move at that depth
- uint32_t findBestMove(struct gameState* gs, int depth);
    - A wrapper that contains depth search.
    - Uses iterative deepening to search from depth 1 up to the requested depth.
    - Parameters:
        - gs: current game state
        - depth: maximum search depth
    - Returns:
        - Best move found within the allotted search time
- void getPseudoLegalMoves(struct gameState* gs, Move* moves, int* moveCount);
    - Generates all pseudo-legal moves for the current player.
    - Does not remove moves that leave the king in check.
    - Parameters:
        - gs: current game state
        - moves: output move array
        - moveCount: output count
    - Returns:
        - Nothing
- void getMoves(struct gameState* gs, Move* moves, int* moveCount);
    - Only generates all possible legal moves.
    - Calls getPseudoLegalMoves and removes moves that leave the current player in check.
    - Parameters:
        - gs: current game state
        - moves: output move array
        - moveCount: output count of legal moves

    - Returns:
        - Nothing
- void applyMove(struct gameState* gs, Move m, struct MoveUndo* u);
    - Applies a move to the board and updates the full game state.
    - Parameters:
        - gs: current game state
        - m: encoded move
        - u: optional undo structure (can be NULL)
    - Returns:
        - Nothing
- void undoMove(struct gameState* gs, const struct MoveUndo* u);
    - Restores the game state after a previous applyMove.
    - Parameters:
        - gs: current game state
        - u: saved undo data
    - Returns:
        - Nothing
- void undoMove(struct gameState* gs, const struct MoveUndo* u);
    - Restores the game state after a previous applyMove.
    - Parameters:
        - gs: current game state
        - u: saved undo data
    - Returns:
        - Nothing
- void initializeBoard(struct piece* board[8][10]);
    - Sets up the initial board position with all pieces in their starting squares.
    - Parameters:
        - board: board array to initialize
    - Returns:
        - Nothing
- void initGameState(struct gameState* state);
    - Initializes the game state for a new game.
    - Sets turn order, counters, castling rights, en passant state, and ant counts.
    - Parameters:
        - state: game state to initialize
    - Returns:
        - Nothing

-
- bool isSquareAttackedBy(struct piece* board[8][10], int row, int col, enum pieceColor attackerColor);
    - Checks whether a given square is attacked by a given side.
    - Parameters:
        - board: current board
        - row, col: target square
        - attackerColor: attacking side
    - Returns:
        - true if the square is attacked
- bool wouldLeaveKingInCheck(struct piece* board[8][10], uint32_t move, enum pieceColor color);
    - Temporarily applies a move and checks whether it leaves that side's king in check.
    - Used by getMoves to filter legal moves from pseudolegal.
    - Parameters:
        - board: current board
        - move: move being tested
        - color: moving side
    - Returns:
        - true if the move is illegal because it exposes the king
- bool isColorInCheck(const struct gameState* gs, enum pieceColor color);
    - Checks whether a specific side is currently in check.
    - Parameters:
        - gs: current game state
        - color: side to test
    - Returns:
        - true if that king is in check
- void getAnteaterMoves(struct piece* board[8][10], int row, int col, uint32_t* moves, int* moveCount);
    - Generates all legal movement patterns for an anteater, including normal adjacent moves and ant-eating capture moves.
    - Parameters:
        - board: current board
        - row, col: anteater location
        - moves: output move array
        - moveCount: output count of generated moves
    - Returns:

- Nothing

3.3 Detailed description of input and output formats

- Syntax or format of a move entered by the user
  - char[] “a2”, char []“a3”
- Syntax or format of a move recorded in the log file
  - char move[] = “a2, a3”, piece.color = black, piece.type = ant,
  - moveNumber = 2, history = {“aa2, aa3”}
- Human readable log file using chess notation
  - ex: “aa2, aa3”

4. Development Plan and Timeline

4.1 Partitioning of Tasks

• main program
• user interface (textual and/or graphics)
• chess objects (data structures for pieces, boards, moves)
• chess rules (possible moves, legal moves)
• lists (or trees) of moves
• strategy (artificial intelligence, AI) module
• log file module
• feature modules
• documentation and testing (everyone do)

4.2 Team member responsibilities

Will - Chess Rules, Main program
Stefano - List of moves, GTK (GUI)
Katrina - Pieces moveset (+ legal moves)
Justin - log file module, Chess objects
Prithvi - feature modules, error handling
Rishab - strategy (Engine) module

Timeline:
Testing and debugging due 4/19
Version 0.5 due 4/20
Version 1.0 due 4/28

Copyright

© 2026 Blunder Boys + Katrina

Error Messages

Error	Meaning
Illegal Move. Please try again.	The entered move violates the rules of chess. Check piece movement and game rules and try again.
Wait for opponent move	Out of turn move
Time warning	Tells how much time left
King is in Check!	Check warning
Cannot castle while in check. King or rook has already moved. Pieces are blocking castling path.	Castling warnings
En passant not available.	Can capture pawn like it has moved one move even if it moved two if the enemy pawn lands next to your pawn after moving two squares in first move.

Index

Ant (Pawn) 3
Ant eating 3
Bishop 3, 4, 8
Castling 3, 5
Check 3, 4, 8
Checkmate 3, 4, 6, 7
En passant 3, 5
King 8
Knight 4, 8
Pawn 3, 4, 8

Queen	4, 5, 8
Rook	3, 4, 8

References

- Chess Programming Wiki. *Negamax*. https://www.chessprogramming.org/Negamax
- Chess Programming Wiki. *Alpha-Beta*. https://www.chessprogramming.org/Alpha-Beta
- Chess Programming Wiki. *Quiescence Search*. https://www.chessprogramming.org/Quiescence_Search
- Chess Programming Wiki. *History Heuristic*. https://www.chessprogramming.org/History_Heuristic
- Chess Programming Wiki. *Zobrist Hashing*. https://www.chessprogramming.org/Zobrist_Hashing
- Youtube. Transposition Tables and Zobrist Keys. https://www.youtube.com/watch?v=QYNRvMolN20.